

NetSage AI — Technical Architecture, Module Breakdown & Technology Stack

1. Technical Architecture

The system will follow a layered architecture:

User Interface Layer

Provides the interface through which the user enters:

- Network symptoms
- Topology notes
- Packet Tracer `show` command outputs
- Troubleshooting case information

Diagnosis Layer

Contains two complementary components:

AI Diagnosis Engine

- Analyzes the supplied symptoms and network evidence.
- Identifies likely root cause.
- Determines the probable OSI layer.
- Assigns a confidence level.
- References the evidence supporting the diagnosis.
- Suggests the next command to execute.
- Provides proposed fix steps.

Rule-Based Checker

- Performs deterministic checks for common networking mistakes.
- Checks duplicate IP addresses.
- Detects incorrect subnet masks.
- Checks gateway mismatches.
- Detects interfaces that are down.
- Checks missing VLANs.
- Checks missing routes.

Human Review Layer

The AI diagnosis is never treated as final automatically.

The reviewer can:

- **Accept** the diagnosis.
- **Edit** the diagnosis.

- **Reject** the diagnosis.

The system records the original AI response and the human decision for responsible-AI analysis.

Verification Layer

After the proposed fix is applied, the network is tested again to determine whether the original problem has been resolved.

Data & Analytics Layer

Stores:

- Troubleshooting cases
- Network evidence
- Expected diagnoses
- AI responses
- Human decisions
- Corrections
- Verification results

The dashboard uses this information to display:

- Issue categories
 - Severity
 - AI versus human agreement
 - Correction statistics
-

2. Module Breakdown

Module 1 — Case Dataset Management

Responsible for creating and storing the minimum 30 troubleshooting cases required by the project.

Each case should contain:

- Case ID
- Symptom
- Topology note
- `show` command outputs
- Expected fault
- OSI layer
- Concept/tag
- Severity

The cases should cover VLAN, gateway, DHCP, DNS, routing, ACL, NAT and wireless issues.

Module 2 — Input and Evidence Processor

Accepts the troubleshooting information and organizes it into a structured format before diagnosis.

Example input:

```
Symptom:
PC receives an IP address but cannot reach the server.

Topology:
PC → Access Switch → Router → Server VLAN

Evidence:
show ip route
show access-lists
show interfaces trunk
```

Module 3 — Rule-Based Network Checker

Runs deterministic checks against the available network information.

Examples:

- Duplicate IP detection
- Wrong subnet mask
- Incorrect default gateway
- Interface shutdown/down
- Missing VLAN
- Missing routing entry

The checker should produce explicit findings that can be compared against the AI diagnosis.

Module 4 — AI Diagnosis Engine

Uses a structured prompt to analyze the case.

The AI output should contain fields such as:

```
{
  "root_cause": "",
  "confidence": "",
  "evidence": [],
  "osi_layer": "",
  "next_command": "",
  "fix_steps": []
}
```

The AI should base its reasoning on the supplied evidence rather than making an unsupported assumption.

Module 5 — Diagnosis Comparison

Compares:

- AI diagnosis
- Rule-checker findings
- Expected diagnosis from the dataset

This provides the basis for evaluating whether the AI identified the correct problem.

Module 6 — Human Review

Allows a human reviewer to classify each AI diagnosis as:

- Accepted
- Edited
- Rejected

For edited or rejected cases, the reviewer records why the AI diagnosis was incorrect or incomplete.

At least five corrected AI responses must be documented.

Module 7 — Fix and Verification

After human approval, the proposed fix can be applied in the Packet Tracer topology.

The case is then tested again to verify whether connectivity or the affected functionality has been restored.

Module 8 — Dashboard

Provides a simple visual summary of the system.

Possible metrics include:

- Number of cases
- Cases by issue type
- Cases by severity
- AI accepted rate
- AI edited rate
- AI rejected rate
- AI-human agreement rate
- Number of AI corrections

Module 9 — Responsible AI Logging

Maintains a record of situations where:

- AI was incorrect.
- AI lacked sufficient evidence.
- Human review changed the diagnosis.

- The final diagnosis differed from the AI suggestion.

This directly supports the project's human-review requirement.

3. Technology Stack

Programming Language

Python

Used for:

- Rule-based network validation
- Data processing
- AI integration
- Case comparison
- Logging
- Dashboard backend where required

AI / LLM Layer

A **large language model API** can be used for the diagnosis engine.

The model will receive structured troubleshooting information and return a structured JSON diagnosis.

Dataset

CSV

The main case dataset will be stored as:

```
cases.csv
```

Prompt Engineering

Prompt templates will be stored separately, including:

```
diagnose_prompt.md
```

Additional helper prompts can also be maintained as separate files.

Dashboard

A lightweight dashboard can be implemented using a Python-based framework such as **Streamlit**, allowing the project to demonstrate:

- Case selection

- Evidence display
- AI diagnosis
- Human review
- Statistics and charts

Networking Environment

Cisco Packet Tracer

Used to:

- Build network topologies.
- Introduce configuration faults.
- Generate symptoms.
- Execute Cisco `show` commands.
- Apply fixes.
- Verify the repaired network.

Data Analysis / Visualization

Python libraries can be used for:

- Data handling
- Statistical summaries
- Dashboard charts

Project Storage Structure

```

NetSage-AI/
|
├── data/
│   └── cases.csv
|
├── prompts/
│   ├── diagnose_prompt.md
│   └── helper_prompts/
|
├── checker/
│   ├── rule_checker.py
│   └── rules/
|
├── ai/
│   └── diagnosis_engine.py
|
├── review/
│   └── human_review.py
|
├── verification/
│   └── verifier.py
|

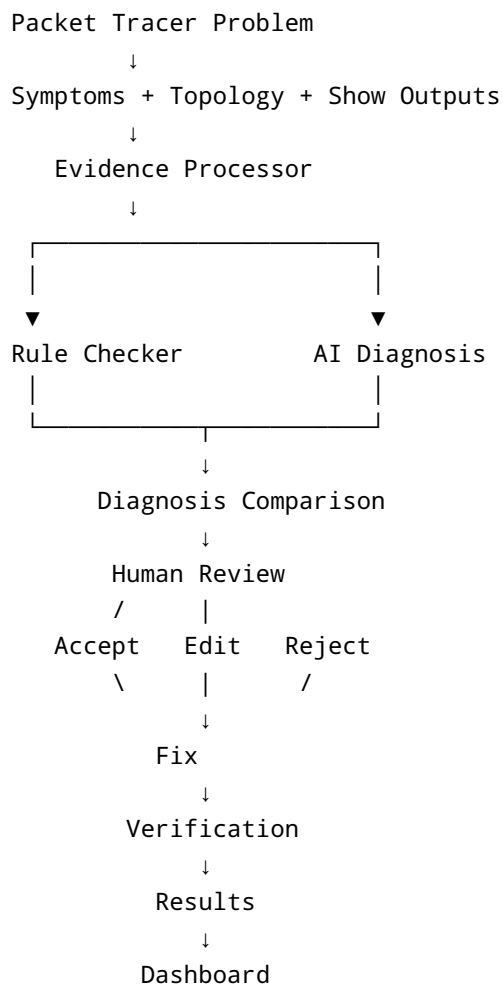
```

```

├── dashboard/
│   └── app.py
├── logs/
│   ├── ai_responses/
│   └── responsible_ai_log.csv
└── README.md

```

4. Overall Flow



The architecture preserves the central requirement of the project: **AI assists with troubleshooting, deterministic rules provide verifiable checks, and a human remains responsible for the final diagnosis and fix.**